



Manual técnico de la App Web SoundFlow

Proyecto integrador: aplicación web SOUNDFLOW

Tecnologías principales: HTML, CSS, PHP, MySQL, XAMPP, API y JSON

Estudiantes: Sebastian Sejas Ureña, Carlos Cespedes Zeballos y Luis Miranda
Maldonado

Carrera: Ing. Sistemas

Licenciado: Toro Ledezma Juan Carlos

Año: 2026

Índice

1. Descripción general de la arquitectura.....	
2. Diagrama de arquitectura.....	
3. Tecnologías y versiones.....	
4. Estructura de carpetas y archivos.....	
5. Modelo de base de datos.....	
6. Diccionario de datos.....	
7. Documentación de la API / endpoints.....	
8. Lógica de negocio relevante.....	
9. Seguridad implementada.....	
10. Pruebas realizadas.....	
11. Limitaciones y mejoras futuras.....	

1. Descripción general de la arquitectura

SoundFlow es una aplicación web para consultar, buscar, reproducir y administrar un catálogo de canciones. El sistema está organizado en tres capas principales: capa de presentación, capa de servicios y capa de datos. Esta organización permite separar la interfaz visual, la lógica de procesamiento y el almacenamiento de información, facilitando el mantenimiento y futuras mejoras del proyecto.

La **capa de presentación** está formada por *index.html*, *assets/css/estilos.css* y *assets/js/app.js*. Esta capa muestra las vistas de inicio, búsqueda, música, biblioteca, playlists, artistas y el formulario de canciones. La **capa de servicios** está representada por *api/canciones.php*, que expone una API REST en PHP. Este archivo recibe las peticiones del navegador, valida los datos, ejecuta las operaciones CRUD y responde en formato JSON. Finalmente, la **capa de datos** utiliza MySQL para almacenar la tabla *canciones*. La conexión a la base de datos se centraliza en *config.php* mediante PDO.

El navegador nunca se conecta directamente a MySQL. Todas las operaciones pasan por la API REST, lo que permite controlar las solicitudes, validar la información enviada y devolver respuestas estructuradas. Los archivos MP3 e imágenes se sirven desde las carpetas locales del proyecto, mientras que la información de las canciones se consulta desde la base de datos y se entrega al frontend en formato JSON.

El flujo principal comienza cuando el usuario abre la aplicación desde el navegador. La interfaz carga los recursos visuales, estilos y scripts necesarios para mostrar el entorno de SoundFlow. Cuando el usuario realiza una acción, como buscar una canción, reproducir un tema o registrar una nueva canción, JavaScript envía una solicitud HTTP hacia la API ubicada en *api/canciones.php*. Esta API funciona como intermediaria entre el frontend y la base de datos, evitando que el usuario tenga acceso directo a las consultas SQL o a la estructura interna de MySQL.

En la capa de servicios, PHP interpreta el tipo de operación solicitada. Por ejemplo, una petición de consulta puede devolver todas las canciones registradas, mientras que una petición de creación puede insertar una nueva canción en la tabla correspondiente. Para conectarse a MySQL se utiliza *config.php*, donde se centralizan los datos de conexión y se emplea PDO como mecanismo de acceso a la base de datos. Esto permite mantener el código más ordenado y facilita la reutilización de la conexión en diferentes operaciones.

La información que devuelve el backend se estructura en formato JSON, ya que este formato es ligero, fácil de interpretar por JavaScript y adecuado para comunicar datos entre cliente y servidor. De esta forma, el frontend puede recibir arreglos u objetos con información como título de la canción, artista, álbum, género, ruta del archivo MP3 e imagen asociada. Con esos datos, JavaScript actualiza la interfaz sin necesidad de modificar directamente la base de datos desde el navegador.

Esta arquitectura también ayuda a que el proyecto sea más fácil de mantener. Si en el futuro se desea cambiar el diseño visual, se trabaja principalmente sobre los archivos de la capa de presentación. Si se desea modificar la forma en que se guardan o consultan las canciones, se ajusta la capa de servicios y la base de datos. Además, si en una versión posterior se decide consumir una API externa de música, esta puede integrarse mediante JavaScript o PHP sin cambiar completamente la estructura principal del sistema.

2. Diagrama de arquitectura

Capa / componente	Archivo o tecnología	Responsabilidad	Comunicación
Usuario	Navegador web	Consulta, busca, reproduce y administra canciones desde la interfaz de SoundFlow.	Interactúa con botones, formularios, buscador y reproductor.
Capa de presentación	index.html, assets/css/estilos.css, assets/js/app.js	Muestra las vistas de inicio, búsqueda, música, biblioteca, playlists, artistas y formulario de canciones.	JavaScript envía solicitudes HTTP mediante fetch y recibe respuestas JSON.
Capa de servicios	api/canciones.php	Expone una API REST en PHP, recibe solicitudes del navegador, valida datos y ejecuta operaciones CRUD.	Recibe peticiones GET/POST y responde al frontend en formato JSON.
Conexión a datos	config.php + PDO	Centraliza la conexión entre PHP y MySQL para mantener el acceso a datos organizado y reutilizable.	Usa PDO para ejecutar consultas SQL hacia la base de datos.
Capa de datos	MySQL en XAMPP	Almacena la tabla canciones con información como título, artista, álbum, género, ruta MP3 e imagen.	No se comunica directamente con el navegador; solo recibe consultas desde PHP.
Archivos locales	Carpetas de imágenes y MP3	Guardan los recursos multimedia que se muestran o	Se sirven desde el proyecto local y se referencian desde la interfaz.

		reproducen en la aplicación.	
Respuesta de datos	JSON	Estructura los datos devueltos por la API para que JavaScript pueda interpretarlos.	Permite actualizar dinámicamente la interfaz de SoundFlow.

Diagrama de arquitectura en Mermaid:

flowchart TD

U[Usuario] --> B[Navegador web]

B --> H[index.html]

H --> JS[assets/js/app.js]

JS -->|fetch HTTP JSON| API[api/canciones.php]

API --> CFG[config.php]

CFG --> PDO[PDO]

PDO --> DB[(MySQL: soundflow)]

JS --> IMG[img/]

JS --> AUDIO[audio/]

API -->|JSON| JS

CHECK[check.php] --> CFG

POSTMAN[Postman] -->|GET POST PUT DELETE| API

PUBLIC[public_api.php] --> CFG

El diagrama de arquitectura representa el flujo real de SoundFlow. El usuario accede desde el navegador web, que carga el archivo *index.html*. Desde allí se ejecuta *assets/js/app.js*, encargado de realizar peticiones HTTP mediante *fetch* hacia la API interna *api/canciones.php*. Esta API procesa operaciones como consultar, crear, actualizar o eliminar canciones, y utiliza *config.php* para obtener la conexión a la base de datos mediante PDO. Finalmente, PDO se comunica con la base de datos MySQL llamada *soundflow*. Además, JavaScript carga recursos multimedia desde las carpetas locales *img/* y *audio/*. El archivo *check.php* permite verificar la conexión de

configuración, *Postman* se utiliza para probar métodos GET, POST, PUT y DELETE, y *public_api.php* también se conecta a la configuración del sistema.

3. Tecnologías y versiones

Tecnología	Uso dentro del proyecto	Versión
HTML	Estructura de las páginas web	HTML5
CSS	Diseño visual, estilos y distribución de la interfaz	CSS3
JavaScript	Consumo de API, interacción con JSON y comportamiento dinámico	ES6 o versión usada
PHP	Backend y conexión con la base de datos	Completar según XAMPP
MySQL	Almacenamiento de usuarios, canciones, artistas y datos relacionados	Completar según XAMPP
XAMPP	Servidor local Apache y MySQL para desarrollo	Completar
API / JSON	Intercambio de datos entre backend, base de datos y frontend	No aplica

4. Estructura de carpetas y archivos

/soundflow

- └── index.html: página principal de la aplicación.
- └── /assets: recursos visuales como imágenes, íconos y archivos multimedia.
- └── /css: hojas de estilo de la interfaz.
- └── /js: archivos JavaScript para interacción y consumo de API.
- └── /api: archivos PHP que reciben solicitudes y devuelven respuestas JSON.
- └── /config: archivos de configuración, como la conexión a la base de datos.

- /includes: componentes reutilizables, funciones o fragmentos de código PHP.
- /db: scripts SQL o respaldo de la base de datos.

La estructura puede adaptarse según los nombres reales usados en el proyecto. Se recomienda mantener los archivos organizados por responsabilidad para facilitar el mantenimiento y futuras mejoras.

5. Modelo de base de datos

La base de datos de SoundFlow se llama *soundflow* y almacena principalmente la tabla *canciones*. Esta tabla guarda la información necesaria para consultar, buscar, reproducir y administrar el catálogo musical de la aplicación.

Campo	Tipo	Restricciones	Descripción
id	INT UNSIGNED	PK, autoincremental	Identificador interno de cada canción.
external_id	VARCHAR(80)	UNIQUE	Identificador externo o local.
titulo	VARCHAR(150)	NOT NULL	Título de la canción.
artista	VARCHAR(150)	NOT NULL	Nombre del artista o artistas.
album	VARCHAR(150)	NULL	Álbum asociado a la canción.
genero	VARCHAR(100)	NULL	Género musical.
duracion	VARCHAR(10)	NULL	Duración visible, por ejemplo 3:17.
imagen_url	VARCHAR(500)	NULL	Ruta o URL de la portada.
audio_url	VARCHAR(500)	NULL	Ruta o URL del archivo de audio.
store_url	VARCHAR(500)	NULL	Enlace externo de tienda, si existe.
fuelle	VARCHAR(50)	DEFAULT local	Origen del registro.
created_at	TIMESTAMP	DEFAULT actual	Fecha de creación.

updated_at	TIMESTAMP	Actualización automática	Fecha de última modificación.
-------------------	-----------	--------------------------	-------------------------------

6. Diccionario de datos

Dato	Tipo esperado	Origen	Requerido	Regla o uso
id	Entero positivo	MySQL o URL	Solo para PUT/DELETE	Identifica un registro existente.
external_id	Cadena	Backend	No en el formulario	Se genera automáticamente con prefijo local_.
titulo	Cadena	Formulario o JSON	Sí	Debe tener al menos 2 caracteres y máximo 150 en la base.
artista	Cadena	Formulario o JSON	Sí	Debe tener al menos 2 caracteres y máximo 150 en la base.
album	Cadena	Formulario o JSON	No	Máximo 150 caracteres en la base.
genero	Cadena	Formulario o JSON	No	Máximo 100 caracteres en la base.
duracion	Cadena m:ss	Formulario o JSON	No	Se usa para mostrar la duración; valor predeterminado 0:00.
imagen_url	Cadena/ruta	Formulario o JSON	No	Usa img/songs/1.jpg si está vacía.
audio_url	Cadena/ruta	Backend o JSON	No	En POST se asigna la ruta del MP3 subido.
store_url	Cadena/URL	Base de datos	No	Enlace externo opcional; queda vacío en altas locales.

fuelle	Cadena	Backend	No	Identifica el origen; las altas del formulario usan usuario.
created_at	Fecha y hora	MySQL	No	Se asigna al crear el registro.
updated_at	Fecha y hora	MySQL	No	Se actualiza automáticamente al modificar.

7. Documentación de la API / endpoints

La API de SoundFlow se encuentra en `/soundflow/api/canciones.php` y permite consultar, crear, actualizar y eliminar canciones. Todas las respuestas se devuelven con *Content-Type: application/json*. Las respuestas correctas incluyen *success: true*, mientras que los errores incluyen *success: false*, un mensaje y el código HTTP correspondiente.

Método	URL	Función	Respuesta habitual
GET	?page=1&search=	Lista canciones paginadas y filtradas.	HTTP 200
GET	?id=1	Consulta una canción por id.	HTTP 200 o 404
POST	URL base	Crea una canción con formulario y MP3.	HTTP 201
PUT	URL base	Actualiza datos usando JSON.	HTTP 200
DELETE	?id=1	Elimina una canción por id.	HTTP 200
OPTIONS	URL base	Responde a solicitudes CORS preliminares.	HTTP 204

GET listado: permite solicitar canciones paginadas y filtradas, por ejemplo *GET /soundflow/api/canciones.php?page=1&search=duki*. La respuesta incluye *success*, un arreglo *data* y metadatos como página actual, cantidad por página, total de registros y número de páginas.

GET por id: consulta una canción específica mediante su identificador, por ejemplo *GET /soundflow/api/canciones.php?id=1*. Si el registro existe, devuelve el objeto de la canción en *data*; si no existe, responde con *success: false* y HTTP 404.

POST crear: crea una nueva canción usando *multipart/form-data*. Los campos principales son *titulo*, *artista*, *album*, *genero*, *duracion* e *imagen_url*. El archivo MP3 debe enviarse con el nombre *audio*. Si la operación es correcta, responde HTTP 201.

PUT actualizar y DELETE eliminar: PUT recibe datos en JSON para actualizar campos editables conservando el identificador del registro. DELETE elimina una canción usando una URL como */soundflow/api/canciones.php?id=1* y devuelve el valor *deleted*, que indica cuántas filas fueron eliminadas. Además, *public_api.php* funciona como API pública de solo lectura y devuelve el catálogo completo, sin implementar POST, PUT ni DELETE.

8. Lógica de negocio relevante

La lógica de negocio de SoundFlow se basa en administrar un catálogo musical desde el navegador. Al cargar la página, *app.js* solicita a la API la primera página del catálogo; la respuesta incluye las canciones y los metadatos de paginación. El sistema muestra las canciones en tarjetas y presenta las primeras cuatro en la vista de inicio.

La búsqueda se realiza mientras el usuario escribe y compara parcialmente los campos *titulo*, *artista*, *album* y *genero* mediante consultas con *LIKE*. La reproducción usa el objeto *Audio* del navegador y toma la ruta del archivo desde *audio_url*,

permitiendo reproducir, pausar, avanzar, retroceder, repetir, usar aleatorio, controlar volumen y progreso.

Los favoritos y playlists se guardan en *localStorage*, por lo que no forman parte del modelo MySQL actual y permanecen solo en el navegador utilizado. Para registrar una canción, el formulario envía datos en *multipart/form-data* junto con un archivo MP3; la API valida título y artista, limita el archivo a 25 MB, acepta solo extensión *.mp3*, guarda el audio en *audio/uploads/* e inserta la ruta en la base de datos.

Como reglas principales, el listado usa 12 canciones por página y la página mínima es 1. El listado general se ordena por *id DESC*, mientras que la consulta pública se ordena por título. Una búsqueda por id inexistente responde HTTP 404. En escritura, PUT actualiza los datos editables, DELETE requiere un id entero positivo y, si falla una inserción, se elimina el archivo MP3 recién subido.

9. Seguridad implementada

- Las consultas de lectura y escritura utilizan sentencias preparadas de PDO para reducir el riesgo de inyección SQL.
- La conexión a la base de datos usa *PDO::ATTR_ERRMODE = PDO::ERRMODE_EXCEPTION* para manejar errores mediante excepciones.
- Se desactiva la emulación de prepared statements, fortaleciendo el uso real de consultas preparadas.
- El frontend escapa los valores antes de construir contenido HTML, evitando mostrar datos sin control en la interfaz.
- El backend valida campos obligatorios como *título* y *artista*, además de comprobar que el archivo subido sea MP3.
- El endpoint *public_api.php* funciona solo en modo lectura y no permite operaciones POST, PUT ni DELETE.

- Los favoritos y playlists guardados en *localStorage* pertenecen al navegador usado y no se sincronizan entre dispositivos.
- Para producción se recomienda cambiar las credenciales por defecto de MySQL y agregar autenticación, autorización, protección CSRF, validación MIME, límites de peticiones y almacenamiento de audio fuera del directorio público.

10. Pruebas realizadas

Prueba	Procedimiento	Resultado esperado	Estado
Ejecución de PHP	Abrir prueba.php desde Apache.	Muestra la versión de PHP.	Realizada.
Conexión a MySQL	Ejecutar check.php.	Confirma conexión a la base soundflow.	Realizada.
Endpoint de listado	Abrir api/canciones.php?page=1.	HTTP 200 y JSON válido.	Realizada.
Carga de la interfaz	Abrir http://localhost/soundflow/.	Se muestra el catálogo sin error de JSON.	Realizada.
GET por id	Probar ?id=1 desde Postman o navegador.	Devuelve una canción.	Preparada en Postman.
POST	Enviar formulario multipart con MP3.	HTTP 201 y nuevo id.	Preparada en la API.
PUT	Enviar JSON con id válido.	HTTP 200 y success: true.	Preparada en Postman.
DELETE	Enviar ?id=id válido.	HTTP 200 y deleted: 1.	Preparada en Postman.
Sintaxis estática	Revisar app.js y api/canciones.php.	Sin errores detectados por el editor.	Realizada.

11. Limitaciones y mejoras futuras

Limitaciones actuales

- No existe autenticación ni gestión de usuarios.

- Favoritos y playlists dependen del *localStorage* de cada navegador y no se sincronizan entre dispositivos.
- La base de datos solo maneja la tabla de canciones y no tiene relaciones para usuarios, playlists o favoritos.
- La subida de archivos valida extensión y tamaño, pero debe reforzarse con validación MIME y controles de contenido para producción.
- La colección de Postman debe actualizarse porque el POST real requiere *multipart/form-data* y un archivo *audio*.
- *index.html* referencia *assets/js/validaciones.js*, pero ese archivo no está presente; se debe crear o consolidar la validación en *app.js*.
- No existen pruebas automatizadas ni pipeline CI/CD.
- La configuración usa *root* sin contraseña, adecuada solo para desarrollo local.

Mejoras futuras

- Implementar usuarios, sesiones y permisos por rol.
- Crear tablas para *usuarios*, *playlists*, *playlist_canciones* y *favoritos*.
- Guardar favoritos y playlists en MySQL para sincronizarlos entre dispositivos.
- Actualizar la colección de Postman para enviar correctamente el POST multipart.
- Crear el archivo de validaciones faltante o eliminar su referencia del HTML.
- Agregar pruebas automatizadas para GET, POST, PUT, DELETE y validaciones de archivos.
- Aplicar autenticación, protección CSRF, límites de peticiones, logs, variables de entorno y HTTPS.
- Agregar eliminación automática del archivo MP3 asociado cuando se borra una canción.